

Project Proposal

Parallel Neural Network Inference in C++

Project Group Number: 2

Group Members:

- Arnar Jónsson
- Changhe You
- Pavlo Shvaiko

Target Application: Parallel Neural Network Inference

We propose to implement inference for a fully connected feed-forward neural network (MLP) in C++. The network will consist of several dense linear layers with ReLU activations, using batched inference in single precision. The main computation is a sequence of matrix multiplications, bias additions, and activation functions. We chose this application because it is suitable for parallelization and it is relevant to the current technology.

Our initial baseline will be a serial C++ implementation of batched MLP inference. We plan to evaluate correctness, profile the main bottlenecks, and estimate the upper bound of achievable speedup based on the fraction of time spent in the compute-intensive kernels. The main performance metrics will include runtime, throughput (samples/second), speedup, parallel efficiency, and scalability on the available three computing systems.

For OpenMP, we will parallelize the compute-intensive parts of inference, mainly the dense layer computations and possibly auxiliary operations such as bias addition and activation. We will study inter-thread overheads, memory bandwidth limitations, and scalability across CPU cores.

For MPI, we plan to implement data-parallel inference by distributing input batches across MPI processes, while replicating model weights on each process. This design keeps the implementation relatively simple and minimizes communication compared to model parallelism. We will evaluate both strong scaling and weak scaling on Dardel and on the school cluster, and model the communication overhead associated with input/output distribution and synchronization.

For the hybrid MPI+OpenMP version, we will combine batch decomposition across MPI processes with shared-memory parallelism inside each process. Finally, we plan to perform optimization guided by profiling, including at least two improvements to the hybrid implementation, and, if feasible, implement OpenMP GPU offloading for comparison with the CPU-based MPI+OpenMP version.

Initial Assessment of Major Challenges

- Designing the MLP implementation so that it remains simple but large enough to demonstrate meaningful speedup.
- Achieving good memory locality and avoiding bandwidth bottlenecks in dense layer computations.
- Modeling the tradeoff between communication overhead and computation for MPI and hybrid execution.
- Finding the best balance between MPI ranks and OpenMP threads for a fixed number of cores.
- If GPU offloading is attempted, managing data movement overhead between host and device.

Tentative Timeline (approximately 6 weeks)

- **Week 1:** Finalize design, choose network sizes and benchmarking setup, implement and verify serial baseline.
- **Week 2:** Baseline profiling and analysis on the available systems; Start OpenMP and MPI parallelization.
- **Week 3:** Finalize OpenMP and MPI parallelization and start combining them together.
- **Week 4:** Benchmarking, analyzing further performance optimizations, bottlenecks.
- **Week 5:** Continue benchmarking and experimenting.
- **Week 6:** Final touches and writing report.

Responsibilities

More detailed tasks will be outlined later and all group members will be contributing equally to all parts.